struct{char;int} 为什么占了 8 字节：对齐与 padding

嵌入式 Linux 手记　2026-11-06　公众号文章 · 原链接 mp.weixin.qq.com

先猜输出。这个结构体占几个字节?

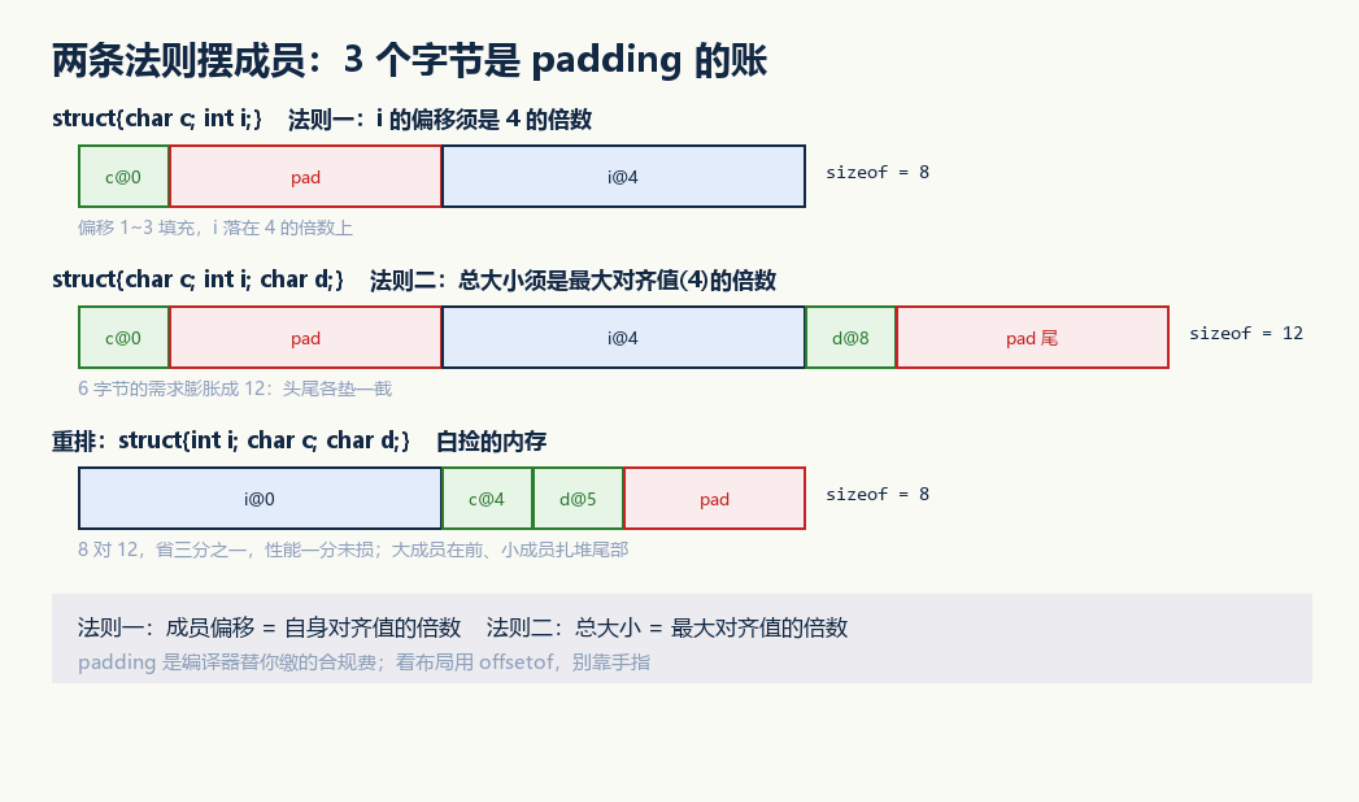
```
struct S {
    char c;   /* 1 字节 */
    int  i;   /* 4 字节 */
};

printf("%zu\n", sizeof(struct S));   /* 手指算: 1 + 4 = 5?  */
```

手指算 5，机器答 8。`char` 和 `int` 之间躺着 3 个"看不见的字节"（padding，填充），它们不归属任何成员，却实实在在计入 `sizeof`。谁偷了这 3 字节？不是编译器手滑，是 CPU 对内存的访问方式决定的。这一篇拆开对齐与填充的全部规则：两条法则算清任何结构体、未对齐访问的硬件代价、`#pragma pack` 的 min 法则，以及一个白捡内存的成员排序技巧。

一、两条法则，算清任何结构体

编译器摆放结构体成员只守两条法则。法则一，成员对齐：每个成员的偏移量必须是"它自身对齐值"的整数倍（对齐值通常就是 `sizeof(成员)`，注意数组成员取的是其元素类型的对齐值而非整个数组的 sizeof；`#pragma pack(n)` 时取 `min(n, sizeof(成员))`）。法则二，整体对齐：结构体总大小必须是"所有成员对齐值的最大者"的整数倍，因为数组里下一个元素的第一个成员也要满足法则一。



图：三条结构体的摆法。法则一决定成员落点，法则二决定尾部补齐；重排成员能白捡三分之一的内存。

拿 `struct S{char c; int i;}` 过一遍：`c` 对齐值 1，放偏移 0；`i` 对齐值 4，最近的合法偏移是 4，偏移 1 到 3 便是填充的 3 字节；总大小当前为 8，已是最大成员（`int` 的 4）的倍数，收工，`sizeof` = 8。再加一个成员 `char d;` 变成 `struct{char c; int i; char d;}`：`d` 放偏移 8，总大小 9 不是 4 的倍数，尾部补 3 字节凑到 12，一个 6 字节的需求膨胀成 12，头尾各垫一截。

二、为什么对齐：CPU 的取货方式

填充是浪费，为什么编译器坚持？答案在硬件。CPU 访问内存按字/块整取，不对齐的地址在很多架构上要么出错要么变慢：早期 ARM、MIPS 上未对齐访问直接触发总线错误（bus error），程序当场死掉；x86 宽容一些，能处理未对齐，但一个跨边界的 `int` 需要两次总线周期才能拼全，性能打折，且不保证原子性。篇 13 讲过"来路不明的内存先过对齐关"，带 MPU 的 MCU 上，内存区域的边界对齐同样是硬件红线（篇 19 的栈保护是同族思路）。对齐不是编译器的洁癖，是硬件的脾气，padding 是编译器替你缴的"合规费"。

三、#pragma pack：min 法则与三笔代价

需要紧凑布局时（网络协议、文件格式、寄存器映射），`#pragma pack(n)` 出场。法则一句话：每个成员的对齐值变成 `min(n, sizeof(成员))`。`#pragma pack(2)` 下 `int` 的对齐值从 4 降到 2，`struct S{char c; int i;}` 变成 6 字节（c@0、垫 1 字节、i@2）；`pack(1)` 则完全紧贴，5 字节需求就是 5 字节。

便宜有三笔账要还。性能：未对齐成员访问变慢（跨架构还可能直接 bus error）；移植：不同编译器对 pack 的默认值与解释有差异，跨平台布局仍可能不同；最隐蔽的是 ABI 风险：两个编译单元对同一个结构体用了不同 pack 设置，字段偏移错位，调用方读到的全是垃圾。对外公开的结构体布局，pack 设置必须全工程一致，或用 `#pragma pack(push, 1)` / `pop` 把影响限定在最小范围。跨机器传结构体更稳的做法是显式序列化：逐字段读写，布局差异根本不参与。



图：pack 的 min 法则与三笔代价。紧凑布局买得，账单是性能、移植与 ABI 一致性。

四、白捡的内存：大成员在前

不动 pack 也有优化空间：调整成员顺序。`struct{char c; int i; char d;}` 是 12 字节，重排成 `struct{int i; char c; char d;}`：`i` @0、`c` @4、`d` @5，总大小 6 补齐到 8：8

对 12，省下三分之一的内存，性能一分未损。法则不变，只是把大成员排前面、小成员扎堆尾部，让填充集中成一个尾巴而不是散落各处。

结构体数量大的系统里这个技巧是白捡的：报文头数组、对象池条目、任务控制块，成员从大到小排一遍，RAM 账单直接变薄。读别人代码时看到"看似随机"的成员顺序，多半不是随手写的，是按对齐法则排的。

五、offsetof：布局规则的官方读数

手指算布局容易错，标准给了官方读数：offsetof 宏（<stddef.h>）返回成员在结构体里的字节偏移，算的就是法则一的结果。它还派生出内核最有名的容器宏 container_of(ptr, type, member)：从成员指针反推出宿主结构体的地址。内核的侵入式链表（list_head 嵌在用户结构体里）能从链表节点跳回宿主对象，靠的全是这个减法：宿主地址 = 成员地址 − 成员偏移。对齐规则决定偏移，offsetof 读出偏移，container_of 用偏移做指针运算，三层一脉相承。

```c
#include <stddef.h>
#include <assert.h>

struct S { char c; int i; };

_Static_assert(offsetof(struct S, i) == 4, "layout changed");
/* container_of 的核心一步：宿主地址 = 成员地址 - offsetof(type, member) */
```

布局敏感的代码还可以在编译期上保险：_Static_assert 让规则被 pack 或编译器变动打破时直接编译失败，错误在编译期而非运行期现形。

padding 还牵出一个邻居坑：结构体不能用 == 比较——C 标准没有为结构体定义 == 运算，写出来直接编译报错；更容易踩的是 memcmp：它逐字节比较，填充字节的值没有定义，两个成员完全相等的结构体也可能被判不等。可靠的写法是逐成员比较，或者 memcmp 之前先把整个结构体 memset 清零，让填充字节有确定的值。

写在最后

1. 两条法则：成员偏移对齐到自身对齐值的倍数，总大小对齐到最大对齐值的倍数；`struct{char;int}` 的 3 字节 padding 就是法则一的账。

2. 对齐的根源是硬件：早期 ARM/MIPS 未对齐直接 bus error，x86 未对齐要两次总线周期且失去原子性；padding 是编译器替你缴的合规费。
3. `#pragma pack(n)` 用 min 法则换紧凑，代价是性能、移植性与 ABI 一致性风险；对外布局用 push/pop 限定范围，跨机器传数据优先显式序列化。
4. 大成员在前是白捡的内存：`{int;char;char}` 比 `{char;int;char}` 小 4 字节，性能无损。
5. `sizeof` 的答案永远是"含 padding 的账单"；看布局用 `offsetof`，别靠手指。

结构体的账是"同一内存怎么摆"，下一篇换一个维度：同一份内存，两台机器怎么读出不同的数。下一篇《大小端：同一段内存，两台机器读出不同的数》，从字节序的历史讲到网络协议与联合体判端序。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。